



四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn

5.4 Syntax-Directed Translation Schemes

SDT

- 语法制导翻译方案SDT

- 在产生式右侧的符号串中嵌入被称为语义动作的程序片段（语义子程序）；
- 语义动作作用一对花括号括起来，在产生式中的位置决定了这个动作的执行时间。

SDD	
1) $T \rightarrow F T'$	$T.val = T'.syn \quad T'.inh = F.val$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val \quad T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

SDT	
1) $T \rightarrow F$	$\{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
2) $T' \rightarrow * F$	$\{ T'_1.inh = T'.inh \times F.val \} T'_1 \{ T'.syn = T'_1.syn \}$
3) $T' \rightarrow \epsilon$	$\{ T'.syn = T'.inh \}$
4) $F \rightarrow \text{digit}$	$\{ F.val = \text{digit.lexval} \}$

What to do ?



How to do ?

在语法分析过程中实现SDT

- 不是所有的SDT都可以在语法分析的过程中实现。(针对的是形成循环依赖的情况)
- 设计翻译方案的原则
 - 必须保证翻译方案中的某个语义动作引用一个属性时，该属性是有定义的。
- 用SDT实现两类重要的SDD
 - 基本文法可以用LR分析，且SDD是S属性的；
 - 基本文法可以用LL分析，且SDD是L属性的。

5.4.1 Postfix Translation Schemes

- 后缀翻译方案
 - 所有语义动作都在产生式最右侧的SDT。
 - 适用于文法可以LR分析且SDD是S属性的。

$$\begin{array}{lll} L & \rightarrow & E \mathbf{n} \quad \{ \text{print}(E.val); \} \\ E & \rightarrow & E_1 + T \quad \{ E.val = E_1.val + T.val; \} \\ E & \rightarrow & T \quad \{ E.val = T.val; \} \\ T & \rightarrow & T_1 * F \quad \{ T.val = T_1.val \times F.val; \} \\ T & \rightarrow & F \quad \{ T.val = F.val; \} \\ F & \rightarrow & (E) \quad \{ F.val = E.val; \} \\ F & \rightarrow & \mathbf{digit} \quad \{ F.val = \mathbf{digit.lexval}; \} \end{array}$$

- Given the input string (a,(a))

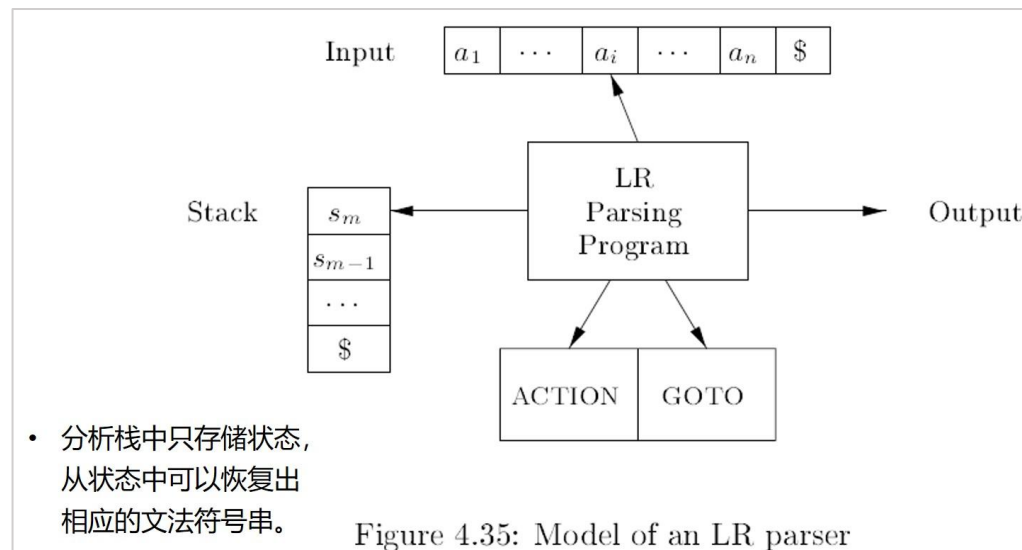
Figure 5.18: Postfix SDT implementing the desk calculator

Postfix SDT (统计括号对数)	
$S' \rightarrow S$	$\text{print}(S.\text{num})$
$S \rightarrow (L)$	$S.\text{num} = L.\text{num} + 2$
$S \rightarrow a$	$S.\text{num} = 0$
$L \rightarrow L_1, S$	$L.\text{num} = L_1.\text{num} + S.\text{num}$
$L \rightarrow S$	$L.\text{num} = S.\text{num}$

5.4.2 Parser-Stack Implementation of Postfix SDT's

- 后缀SDT可以在LR语法分析过程中实现。
 - 分析过程中，归约某个产生式时，执行产生式最右侧的语义动作。
 - LR分析器的栈中存放：状态 / 文法符号。

	State	Symbol	Input	Parsing Action	Semantic Action
1		\$	$i_1 * i_2 + i_3 \$$	shift	
2		$\$i_1$	$*i_2 + i_3 \$$	reduce by $F \rightarrow i$	
3		$\$F$	$*i_2 + i_3 \$$	reduce by $T \rightarrow F$	
4		$\$T$	$*i_2 + i_3 \$$	shift	
5		$\$T*$	$i_2 + i_3 \$$	shift	
6		$\$T*i_2$	$+i_3 \$$	reduce by $F \rightarrow i$	
7		$\$T*F$	$+i_3 \$$	reduce by $T \rightarrow T*F$	
8		...	...	...	



分析栈和语义动作的变化

- 分析栈中存放：状态 / 文法符号，属性值。
- 把语义动作进一步改写成具体可执行的栈操作。

$$A \rightarrow XYZ \{ A.a = f(X.x, Y.y, Z.z) \}$$

$\{ stack[top-2].val = f(stack[top-2].val, stack[top-1].val, stack[top].val); \quad top = top-2; \}$

	X	Y	Z
	X.x	Y.y	Z.z

↑
top

状态 / 文法符号
属性值

	A		
	A.a		

↑
top

Figure 5.19: Parser stack with a field for synthesized attributes

Example 5.15

$L \rightarrow E \mathbf{n}$ { $\text{print}(\text{stack}[\text{top} - 1].\text{val}); \quad \text{top} = \text{top} - 1; \}$

$E \rightarrow E_1 + T$ { $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} + \text{stack}[\text{top}].\text{val}; \quad \text{top} = \text{top} - 2; \}$

$E \rightarrow T$

$T \rightarrow T_1 * F$ { $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} \times \text{stack}[\text{top}].\text{val}; \quad \text{top} = \text{top} - 2; \}$

$T \rightarrow F$

$F \rightarrow (E)$ { $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 1].\text{val}; \quad \text{top} = \text{top} - 2; \}$

$F \rightarrow \mathbf{digit}$

	T_1	*	F
	$T_1.\text{val}$		$F.\text{val}$

			digit
			$\text{digit}.\text{val}$

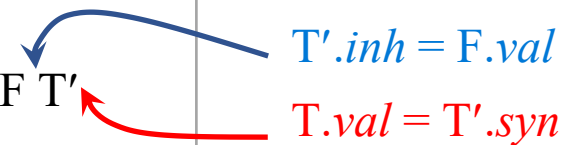
	T		
	$T.\text{val}$		

			F
			$F.\text{val}$

	symbol	Input	Parsing Action	Semantic Action	Value Stack	Stack Manipulation
1	\$	3*5+4n	shift		3	
2	\$3	*5+4n	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$	3	
3	\$F	*5+4n	$T \rightarrow F$	$T.val = F.val$	3	
4	\$T	*5+4n	shift		3	
5	\$T*	5+4n	shift		3 5	
6	\$T*5	+4n	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$	3 5	
7	\$T*F	+4n	$T \rightarrow T * F$	$T.val = T_1.val \times F.val$	15	print(stack[top-2].val);top -= 2;
8	\$T	+4n	$E \rightarrow T$	$E.val = T.val$	15	
9	\$E	+4n	shift		15	
10	\$E+	4n	shift		15 4	
11	\$E+4	n	$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$	15 4	
12	\$E+F	n	$T \rightarrow F$	$T.val = F.val$	15 4	
13	\$E+T	n	$E \rightarrow E + T$	$E.val = E_1.val + T.val$	19	
14	\$E	n	shift		19	
15	\$En		$L \rightarrow En$	print(E.val)	19	print(stack[top-1].val); top--;
16	\$L					

5.4.5 SDT's for L-Attributed Definitions

- L-SDD (基本文法可以用LL分析) 到 SDT的转换规则
 - 将计算非终结符A的继承属性的动作放在产生式中紧靠A之前的位置;
 - 将计算产生式头的综合属性的动作放在产生式的最右侧。
- **Example 5.3**

Production	Semantic Rules	Actions
1) $T \rightarrow F T'$ 	$T'.inh = F.val$ $T.val = T'.syn$	$T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
2) $T' \rightarrow * F T_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$	$T \rightarrow * F \{ T_1'.inh = T'.inh \times F.val \} T_1' \{ T'.syn = T_1'.syn \}$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$	$T' \rightarrow \epsilon \{ T'.syn = T'.inh \}$
4) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$	$F \rightarrow \mathbf{digit} \{ F.val = \mathbf{digit.lexval} \}$

Example 5.19

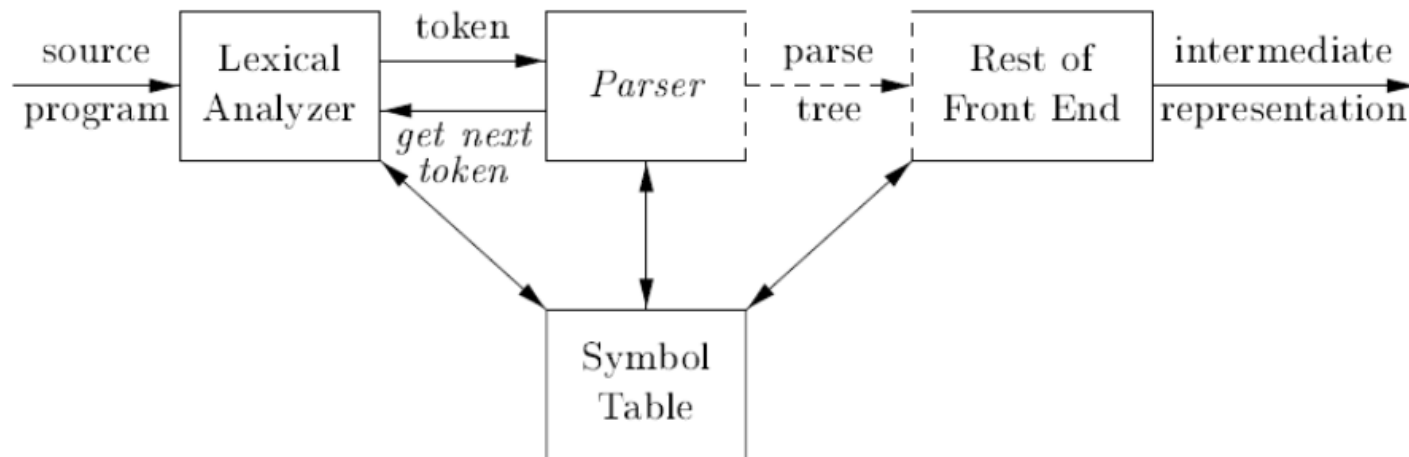
- $S \rightarrow \mathbf{while} (C) S_1$
 $L1 = new();$
 $L2 = new();$
 $S_1.next = L1;$
 $C.false = S.next;$
 $C.true = L2;$
 $S.code = \mathbf{label} \parallel L1 \parallel C.code \parallel \mathbf{label} \parallel L2 \parallel S_1.code$

Figure 5.27: SDD for while-statements

5.5 Implementing L-Attributed SDD's

- S-SDD is suitable for LR-parsing, while L-SDD is suitable for LL parsing. (“适用于” ≠ “只能用于”)

Implementation

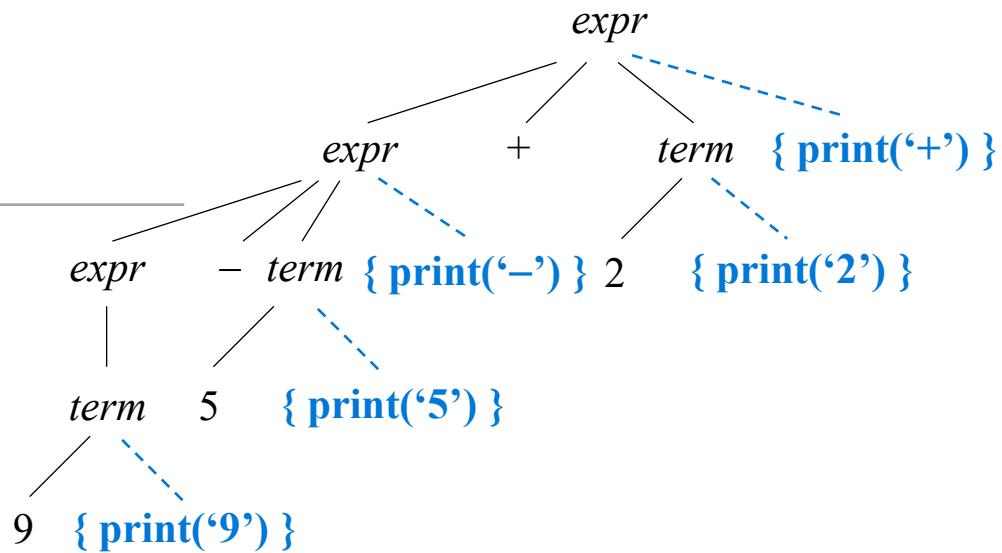


- 在语法分析（递归下降 / LL / LR）的过程中实现SDT。
- 在树的遍历过程中实现SDT (2.3.4, 2.3.5, 2.5.2)
 - 建立语法分析树，为每个语义动作构造一个额外的子结点；
 - 按照深度优先，从左到右的顺序遍历语法分析树。如果需要的话，可多次遍历。

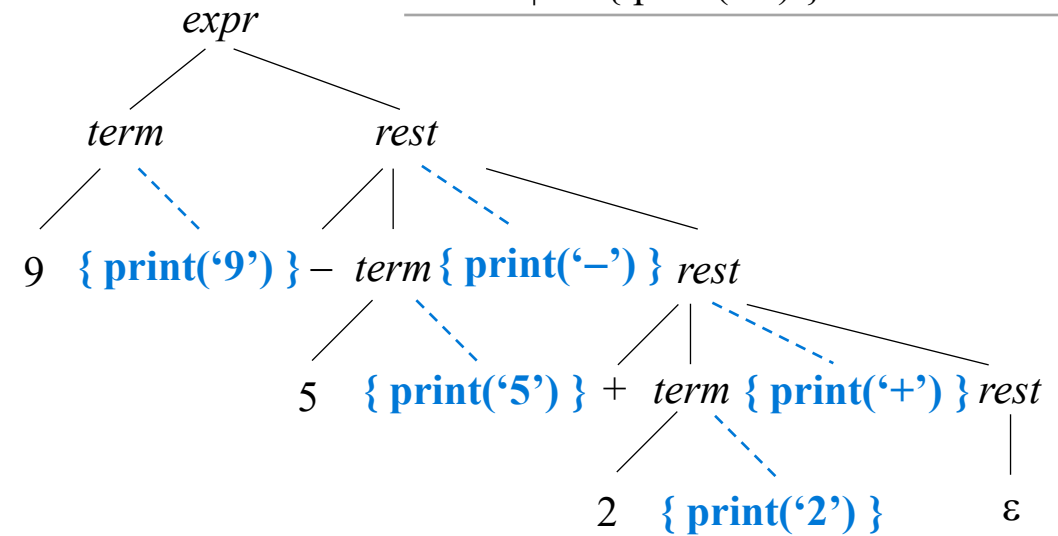
Example 2.12 & 2.13

- Translation of $9-5+2$ to $95-2+$

$expr \rightarrow expr_1 + term \{ \text{print}('+') \}$
 $expr \rightarrow expr_1 - term \{ \text{print}('-') \}$
 $expr \rightarrow term$
 $term \rightarrow 0 \{ \text{print}('0') \}$
 $\dots$
 $| 9 \{ \text{print}('9') \}$



$expr \rightarrow term \text{ rest}$
 $rest \rightarrow + term \{ \text{print}('+') \} rest$
 $| - term \{ \text{print}('-') \} rest$
 $| \epsilon$
 $term \rightarrow 0 \{ \text{print}('0') \}$
 $\dots$
 $| 9 \{ \text{print}('9') \}$



5.5.1 在递归下降分析过程中进行翻译

- 每个非终结符A都有一个函数A，以参数和返回值的形式处理属性：
 - 函数A的参数是非终结符A的继承属性；
 - 函数A的返回值是非终结符A的综合属性的集合。

$exp \rightarrow term \{ addop \ term \}$

```
function exp: integer;  
var temp: integer;  
begin  
  temp:=term;  
  while token = +|- do  
    match(token);  
    case token of  
      +: temp:=temp+term;  
      -: temp:=temp-term;  
    end case;  
  end while;  
  return temp;  
end exp;
```

$exp \rightarrow term \ exp'$
 $exp' \rightarrow addop \ term \ exp' \mid \epsilon$

$E \rightarrow TE'$	$E.val = E'.syn$ $E'.inh = T.val$
$E' \rightarrow +TE_1'$	$E_1'.inh = E'.inh + T.val$ $E'.syn = E_1'.syn$
$E' \rightarrow \epsilon$	$E'.syn = E'.inh$

```
exp() {  
  term();  
  exp_prime();  
}  
  
exp_prime() {  
  if lookahead is addop:  
    match(addop);  
    term();  
    exp_prime();  
  else:  
    return;  
}
```

End of Chapter Five